

Failure (search fails mid-run) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `search_failure_practice.py`: a checkpointer's whole promise is that a crash doesn't erase earlier progress — this exercise proves it on purpose, before the Build Task's search node needs that guarantee for real. **If not:** the Build Task's retry loop would be built on an untested assumption about what actually survives a mid-run failure.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# search_failure_practice.py
from langgraph.checkpoint.memory import MemorySaver

def search_node(state):
    # temporary, for this test
    raise RuntimeError("simulated vector store outage")

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "failure-test-1"}}

try:
    graph.invoke({"task": "What does the policy say about refunds?"}, config)
except RuntimeError as e:
    print("Caught expected failure:", e)

snapshot = graph.get_state(config)
print("Surviving state:", snapshot.values)
print("Was about to run:", snapshot.next)
```

This confirms state survives the crash and prints it. It doesn't assert anything specific about that state's correctness — just prints it for a human to eyeball.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — real assertions, and a test that fails loudly if the crash stops happening

```
# search_failure_practice.py
from langgraph.checkpoint.memory import MemorySaver

def search_node(state: dict) -> dict:
    # temporary, for this test
    raise RuntimeError("simulated vector store outage")

def test_state_survives_search_failure() -> None:
    checkpointer = MemorySaver()
    graph = builder.compile(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "failure-test-1"}}

    try:
        graph.invoke(
            {"task": "What does the policy say about refunds?"}, config
        )
        raise AssertionError("expected the graph to raise, but it didn't")
    except RuntimeError as e:
        print(f"Caught expected failure: {e}")

    snapshot = graph.get_state(config)
    assert "task" in snapshot.values, "the task should still be in state"
    assert snapshot.next == ("search_node",), "should stop before search_node"
    print("Confirmed: state is intact, graph knows where it stopped.")

test_state_survives_search_failure()
```

Difference from Basic: full type hints. Wrapping the test in a function with real assertions, not just prints — this is the difference between "I looked at it and it seemed fine" and a test you can rerun automatically after any future change. The extra `raise AssertionError` if the graph *doesn't* fail is a safety check — without it, a bug that accidentally makes `search_node` stop raising would make this whole test silently pass without testing anything. This version still gives up completely on the first failure — no retry — which is what Approach 2 changes.

Approach 2 — a retry with backoff, transient errors only

Story: not every failure deserves the same response — a vector store down for 200ms is nothing like one down for an hour, or a malformed query that fails identically on every retry. A node that just re-raises on any exception treats all three the same way. **If not:** the Build Task's search step would fail an entire user-facing run on a single, brief network blip a simple retry could have absorbed.

```
# search_failure_practice.py
import logging
import time

logger = logging.getLogger(__name__)

TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3

def search_node(state: dict) -> dict:
    for attempt in range(MAX_ATTEMPTS):
        try:
            results = retrieve(state["task"])
            return {"sources": results}
        except TRANSIENT_ERRORS as exc:
            if attempt == MAX_ATTEMPTS - 1:
                raise
            wait_seconds = 2 ** attempt
            logger.warning(
                "Search attempt %d failed (%s); retrying in %ds",
                attempt + 1, exc, wait_seconds,
            )
            time.sleep(wait_seconds)
    # unreachable: the loop above always returns or raises
    raise RuntimeError("search_node exited its retry loop unexpectedly")
```

A `TimeoutError` or `ConnectionError` gets up to `MAX_ATTEMPTS` tries, with a growing delay between them (1s, then 2s), so a brief network blip is absorbed automatically. Anything else — a malformed query, a programming bug — is not in `TRANSIENT_ERRORS`, so it propagates immediately on the first attempt; retrying a bug five times just wastes five times as long finding out it still doesn't work.

`MAX_ATTEMPTS` is written here as a plain module constant so the retry logic is easy to follow — in a real deployment, a retry count like this is exactly the kind of value that belongs in `config.py` (loaded from `.env`, per Doc01) instead of a hardcoded literal, since how aggressively to retry is an operational tuning knob, not something tied to the code's logic.

Approach 3 — retry plus proving both outcomes with a real test

Story: Approach 2 is the retry logic itself; this approach tests it the way `search_failure`'s own lesson demands — proving both that a transient failure recovers, and that state still survives when retries are genuinely exhausted. **If not:** a retry loop with no test could silently start retrying permanent failures, or stop retrying transient ones, without anyone noticing until it matters.

```
# search_failure_practice.py
import time
from langgraph.checkpoint.memory import MemorySaver

TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3

def make_search_node(fail_times: int, permanent: bool = False):
    """Test helper: a search_node that fails `fail_times` before succeeding."""
    calls = {"count": 0}

    def search_node(state: dict) -> dict:
        for attempt in range(MAX_ATTEMPTS):
            calls["count"] += 1
            if calls["count"] <= fail_times:
                if permanent:
                    # not transient -- no retry
                    raise ValueError("malformed query")
                    raise TimeoutError("simulated timeout")
                return {"sources": ["a real result"]}
            raise RuntimeError("exhausted retries")

    return search_node

def test_recovers_from_transient_failure() -> None:
    node = make_search_node(fail_times=2) # fails twice, then succeeds
    result = node({"task": "test"})
    assert result["sources"] == ["a real result"]
    print("Recovered after 2 transient failures, as expected.")

def test_permanent_failure_does_not_retry() -> None:
    node = make_search_node(fail_times=1, permanent=True)
    try:
        node({"task": "test"})
        raise AssertionError("expected a ValueError, but none was raised")
    except ValueError:
        print("Permanent failure raised immediately, with no retry delay.")
```

```
def test_state_survives_when_retries_are_exhausted() -> None:
    checkpointer = MemorySaver()
    graph = builder.compile(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "exhausted-retries-test"}}

    try:
        graph.invoke({"task": "something needing search"}, config)
        raise AssertionError("expected the graph to eventually raise")
    except TimeoutError:
        pass

    snapshot = graph.get_state(config)
    assert "task" in snapshot.values
    print("Even with retries exhausted, pre-failure state is still intact.")

test_recovers_from_transient_failure()
test_permanent_failure_does_not_retry()
test_state_survives_when_retries_are_exhausted()
```

Difference from Approach 1, and between Approaches 2/3: Approach 1 proves the checkpointer's guarantee once, against a node that fails exactly once and stays failed — a correct but incomplete picture of a real deployment, where most external failures are brief. Approach 2 adds the retry logic itself: transient errors get a few chances with growing delays, permanent errors fail immediately. Approach 3 doesn't change the retry logic at all — it proves all three outcomes a reviewer would actually want confirmed: recovery after a transient blip, no wasted retries on a permanent failure, and Approach 1's original "state survives" guarantee still holding even when every retry is used up.

Which one should you actually write? Approach 1's plain crash-and-check test is worth keeping regardless — it's the simplest proof the checkpointing promise holds at all, and it's what `search_failure`'s Core Concepts point is really about. Add Approach 2's retry loop the moment your search backend is a real network service instead of a local file or in-memory store — a single dropped connection shouldn't fail an entire user-facing run. Approach 3's three-part test suite is worth writing once the retry logic itself is something other people (or future you) will change — a retry loop with no test can silently start retrying permanent failures, or stop retrying transient ones, without anyone noticing until it matters.